Module 4 — Laboratory: Your First Real Database

Vistula University · IT Fundamentals · Laboratory assignment

Vistula University · IT Fundamentals

Estimated Time

60–90 minutes.

Prerequisites

- Module 3 lab is done: your Student Task Manager (`tasks.html`) works and saves tasks in localStorage.
- Your **teacher** gives you:
- the **Supabase project URL** (looks like `https://xxxxx.supabase.co`),
- the **publishable anon key** (a long text string).

> **Important:** You do **not** create your own Supabase project. Everyone uses the class project the teacher prepared. Ask the teacher if something does not work.

You work in pairs (as in earlier modules). Push your work to your own repository and ask your partner to review it.

Task 1 — Create the `tasks` Table (15 min)

Your teacher gives you access to the Supabase **SQL Editor** (in the Supabase dashboard). Paste this SQL and run it:

```
-- The tasks table for the Student Task Manager
CREATE TABLE tasks (
  id         bigserial PRIMARY KEY,
  user_id    uuid,
  title      text,
  done       boolean DEFAULT false,
  created_at timestamptz DEFAULT now()
);
```

What the columns mean:

- `id` — the **primary key**. `bigserial` means: a unique number, created automatically for every new row.
- `user_id` — the **foreign key**. It points to the logged-in user. `uuid` is a long random text id.
- `title` — the task text.
- `done` — true or false. Default is `false` (not done yet).
- `created_at` — the time when the row was created. `now()` fills it in automatically.

Enable Row Level Security

Now add the safety rules. Still in the SQL Editor, run:

```
-- Turn on row-level security for this table
ALTER TABLE tasks ENABLE ROW LEVEL SECURITY;

-- Rule: a logged-in user can see and change ONLY their own rows
CREATE POLICY "users manage own tasks"
ON tasks FOR ALL
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id);
```

In words, this policy says: *"A row belongs to you only when `user_id` is your user id (`auth.uid()`). You may select, insert, update, and delete such rows — and nothing else."*

Check yourself

- Open **Table Editor** in Supabase. Can you see the `tasks` table?
- Can you see the policy under **Authentication** → **Policies** (or the table's policy list)?

Task 2 — Connect the Page to Supabase (20 min)

2.1 Include the Supabase library

In `tasks.html`, add this line **before** your own `<script>`:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

`supabase-js` is a small library. It gives JavaScript functions that talk to the Supabase database for you.

2.2 Create the client

At the top of your script, replace the old localStorage code:

```
// Use the URL and key your teacher gave you
const SUPABASE_URL = "https://xxxxx.supabase.co";   // <-- your project URL
const SUPABASE_KEY = "eyJhbGciOi...";               // <-- your anon key

const supabase = window.supabase.createClient(SUPABASE_URL, SUPABASE_KEY);
```

> The anon key is called "publishable". It is safe to put in a web page, **because** Row Level Security protects the data. Without RLS, this key would let anyone read and change everything. That is why Task 1 mattered.

2.3 Read — show the tasks (SELECT)

```
async function loadTasks() {
  const { data, error } = await supabase
    .from("tasks")
    .select("*")
    .order("created_at", { ascending: false });
```

```
  if (error) { alert("Could not load tasks: " + error.message); return; }
  renderTasks(data); // your function that draws the list
}
```

Supabase builds the SQL `SELECT * FROM tasks ...` for you.

2.4 Create — add a task (INSERT)

```
async function addTask(title) {
  const { error } = await supabase
    .from("tasks")
    .insert({ title: title }); // user_id is filled in by the server for a logged-in user

  if (error) { alert("Could not add task: " + error.message); return; }
  loadTasks();
}
```

2.5 Update — toggle "done" (UPDATE)

```
async function toggleDone(id, newValue) {
  const { error } = await supabase
    .from("tasks")
    .update({ done: newValue })
    .eq("id", id); // WHERE id = ...

  if (error) { alert("Could not update task: " + error.message); return; }
  loadTasks();
}
```

2.6 Delete — remove a task (DELETE)

```
async function deleteTask(id) {
  const { error } = await supabase
    .from("tasks")
    .delete()
    .eq("id", id);

  if (error) { alert("Could not delete task: " + error.message); return; }
  loadTasks();
}
```

Check yourself

- Open `tasks.html`, log in (your teacher will tell you how accounts work in this project), and add a task.
- Refresh the page. Is the task still there? Now open the same page on your **phone** or another browser. The same task is there — this is why localStorage was not enough!

Task 3 — Why Unrestricted Public Access Must Not Happen (10 min, discussion)

Imagine we had run the SQL **without** Row Level Security. The `tasks` table would allow anyone with the anon key — that means **everyone on the internet** — to do anything:

- **Read:** anyone could see every user's private tasks.
- **Write:** anyone could insert fake tasks into other people's lists.
- **Update:** anyone could mark other people's tasks as done.
- **Delete:** one bad request could delete **every row** of **every user**.

The anon key is not a secret — it ships inside your web page. So the database itself must be the guard. RLS makes the database check, for **every single row**, that the logged-in user owns that row before it allows anything. Your JavaScript is a convenience; the policy is the real lock.

Discuss with your partner: which of the four CRUD actions is most dangerous without RLS, and why?

Task 4 — Branch, Pull Request, Review, Merge (20 min)

Work exactly as in Module 3:

1. Create a feature branch: `git checkout -b module4-supabase`
2. Commit your changes with a clear message, for example: `Connect tasks to Supabase with RLS`.
3. Push the branch to GitHub.
4. Open a **pull request** (PR) on GitHub.
5. Ask your **partner** to review the PR. The partner checks: - no localStorage code for tasks remains, - the URL/key are the class ones, - every CRUD action works.
6. After approval, **merge** the PR into `main`.

Expected Deliverables

- A `tasks.html` (plus your JS) connected to Supabase: Create, Read, Update, Delete all work against the database.
- The `tasks` table with RLS enabled and the "users manage own tasks" policy.
- A merged pull request reviewed by your partner.

Submission Instructions

Submit through the course platform on the assignment page (Assignments section): paste your GitHub repository URL / deployed app URL / PR URL as required.

1. **Repository URL** (your GitHub repo, with the merged PR visible).
2. **Page URL** (where your app is published, e.g. GitHub Pages).
3. **Names and student IDs** of both partners.

One submission per pair is enough — list both people.